



Санкт-Петербургский государственный университет

Кафедра системного программирования

Разработка образовательного комплекса виртуального устройства и минимальной ОС на архитектуре RISC-V

Кисельков Денис Андреевич, Группа 24М.71-мм

Отчет по учебной практике в форме «Производственное задание»

Научный руководитель: к.ф.-м.н. Д.В. Луцив, доцент кафедры системного программирования

Рецензент: К.А. Жиданов, Ведущий инженер-программист, ООО «Специальный
Технологический Центр»

- Политика импортозамещения, субсидирование разработки ПО и санкционные ограничения стимулируют переход на российские решения и увеличивают долю Linux.
- Доля ПО в ИТ-отрасли достигла 40%, аппаратная составляющая сокращается; растёт спрос на разработчиков системного уровня.
- Рынок труда фиксирует дефицит: 1 млн ИТ-вакансий, 5,3 тыс. специалистов по микроэлектронике; более 60% позиций не закрыты из-за пробелов в системном мышлении выпускников.
- Университеты преподают архитектуру ЭВМ и ОС фрагментарно: студенты не видят, как из вентиляей собирается процессор, как цикл ОС реализуется в машинном коде, как системные абстракции работают в ядре.
- Ценные открытые проекты вроде nand2tetris и xv6 демонстрируют системный подход, но остаются вне образовательных программ и труднодоступны для начинающих.

Цель и задачи

Цель — разработка учебного программного комплекса, объединяющего виртуальный RISC-V-процессор, минимальную UNIX-подобную ОС и легковесный компилятор языка C для сквозной демонстрации жизненного цикла программ.

Задачи

- Провести обзор открытых архитектур, легковесных UNIX-подобных ОС и компактных C-компиляторов; обосновать технологический стек.
- Спроектировать архитектуру комплекса, взаимодействие Verilog-модели с периферией, ОС и компилятором, средства отладки.
- Реализовать на SystemVerilog синтезируемое ядро RISC-V RV32I с полной поддержкой целочисленной арифметики и трассировкой.
- Адаптировать и собрать ОС хвб для созданного процессорного ядра.
- Адаптировать легковесный компилятор TinyCC для работы в гостевой ОС.

Обзор: Архитектуры процессоров

Рассматривались открытые RISC-архитектуры, пригодные для образовательного стенда.

- MIPS: десятилетия служил академическим целям, но развитие открытых вариантов остановлено, современные реализации требуют коммерческих лицензий.
- ARM: проприетарная архитектура, свободное использование ограничено лицензионными соглашениями.
- RISC-V: свободная ISA, модульная структура, базовое подмножество RV32I сочетает компактность с полноценными RISC-принципами; развитая экосистема (GCC, LLVM, эмуляторы).

Операционная система (ОС) — это комплекс программного обеспечения, который управляет аппаратными ресурсами компьютера и предоставляет интерфейс для взаимодействия между пользователем и устройством. Она выполняет множество ключевых функций

- Компиляция и исполнение программ
- Управление процессором
- Контроль памяти
- Организация файловой системы
- Управление устройствами ввода-вывода
- Предоставление интерфейса пользователя

Обзор: сравнение операционных систем

Критерии ОС	Доступность	ЯП	Документация	Размер (тыс)
Первая версия ядра Linux	GNU GPL	C/asm	частичная	10
Старые GNU/Linux	GNU GPL	C/asm	частичная	>200
GNU/Mes	GNU GPL	C/asm	планируется полная	>10
Embox	BSD	C	частичная	60
Zephyr	closed	C	частичная	>1000
CP/M	MIT	C/asm	частичная	>10
Unix System V6	closed	C/asm	частичная	40
MicroDOS	closed	C/asm	частичная	
Minix	BSD	C	полная	30
uClinux	GPL	C	частичная	>200
xv6	MIT	C	полная	9

Учебные компьютеры представляют собой виртуальные устройства, предназначенные для образовательных целей. Эти компьютеры демонстрируют часть принципов архитектуры ЭВМ и позволяют изучить их без необходимости приобретения физических устройств.

Основные критерии учебного компьютера

- Набор инструкций и архитектура
- Наличие и тип компонентов
- Способ реализации
- Интерфейс и интерактивность

Обзор: сравнение учебных компьютеры

Критерии УК	Команды	Прерывания	Многоядерность	Переферия
LMC	10	-	-	-
SIC/XE	>50	+	-	+
Нейман	10	-	-	+
E14	34	+	+	+
Макет ЭВМ (CDROM) ¹	>50	+	-	+
RISC-V RV32IM	>50	+	-	+
Nand2Tetris	>100	+	-	+
Модель ЭВМ	46	+	-	+

¹Макет ЭВМ (CDROM) - у проекта закрыт исходный код

- **Аппаратная часть:** SystemVerilog, Icarus Verilog, GTKWave.
- **Системное ПО:** ANSI C, ассемблер RISC-V.
- **Кросс-компиляция:** riscv-gnu-toolchain.
- **Вспомогательные скрипты:** Python 3 (ассемблер encode.py, сравнение логов, генерация hex).
- **Сборка и контейнеризация:** Makefile, Docker, Git-репозиторий.

Архитектура комплекса

Четыре взаимодействующих уровня:

- **Инструментальный:** riscv-gnu-toolchain, скрипты ELF→hex, Makefile, Docker.
- **Моделируемый (Verilog):** top.sv → RV32ICPU, память, UART, CLINT, RAM-диск, загрузчик.
- **Операционный:** конфигурации run_c_version (C-программа без ОС) и xv6_version (ядро xv6 с ФС).
- **Наблюдательный:** генерация VCD, сравнение лога UART с эталоном QEMU, GTKWave.

Базовые решения: безконвейерное ядро, адресное пространство QEMU-virt, автоматический консольный ввод-вывод через файлы, два режима выполнения, контейнеризация.

Моделируемая платформа

Адресное пространство, совместимое с QEMU virt:

- RAM: 0x80000000 – 0x8FFFFFFF (128 МБ), инициализация `$readmemh`.
- UART: 0x10000000 – 0x1000000F, эмуляция 16550-совместимого приёмопередатчика.
- CLINT: 0x02000000 – 0x0200FFFF, счётчик `mtime` и регистр `mtimestr`, прерывание по совпадению.
- RAM-диск: 0x20000000 – 0x20100000 (1 МБ), образ файловой системы `xv6`, доступ без `VirtIO`.

Загрузчик: `$readmemh` заполняет RAM и диск; сброс устанавливает `PC = 0x80000000`.

Два режима: `run_c_version` (только UART) и `xv6_version` (добавлены CLINT и RAM-диск).

Реализация процессорного ядра RV32I

- Безконвейерное синтезируемое ядро, 1 инструкция за такт.
- **Модульный состав:** ProgramCounter, PCUpdate, InstructionMemory, ControlUnit, ImmediateGenerator, RegisterFile, ALUControl, ALU, BranchComparator, DataMemory.
- **Полная поддержка RV32I:** 47 инструкций, включая ECALL/EBREAK.
- **Отладочные средства:** \$display потактового состояния, генерация VCD-файла, просмотр в GTKWave.
- **Ассемблер encode.py:** трансляция текстовых программ с метками и псевдоинструкциями в hex.

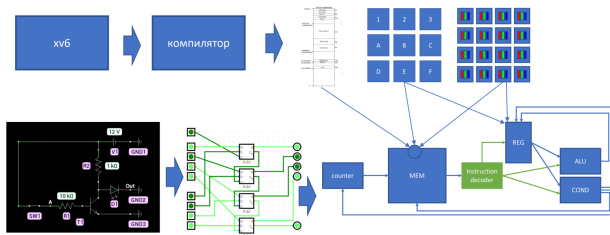
Адаптация xv6 и интеграция TinyCC

- Портирован xv6-rv32: заменены управление памятью (Sv32), ассемблерный слой (entry.S, swtch.S, trampoline.S), обработка прерываний (M-mode), драйвер диска.
- Драйвер диска: виртуальный диск VirtIO заменён прямым копированием блоков из RAM-диска; буферный кэш и файловая система сохранены без изменений.
- Ядро собирается тулчейном riscv64-unknown-elf-gcc, образ ФС формируется утилитой mkfs.
- TinyCC: портирован на RV32I как пользовательская утилита, не используется для сборки стенда. Внутри xv6 компилирует и выполняет C-программы на том же виртуальном процессоре.

Тестирование и верификация

- Трёхуровневая методика: тесты RV32I, C-программы без ОС, проверка хвб.
- Все 47 инструкций RV32I проверены; состояние регистров совпало с эталонным ISA-симулятором.
- C-программы: программы факториала и Фибоначчи дали ожидаемый вывод UART.
- хвб: успешная загрузка, работа shell, ls, cat, echo, mkdir, rm; чередование фоновых процессов по таймеру (подтверждено VCD).
- TinyCC внутри хвб успешно скомпилировал и выполнил пользовательскую программу.

Полное устройство компьютера



Лекция "От транзисторов к операционным системам"

Заключение

Для достижения цели выпускной квалификационной работы были получены следующие результаты

- Обоснован стек RISC-V RV32I / xv6 / TinyCC, спроектирована модульная архитектура.
- Реализовано синтезируемое безконвейерное ядро RV32I с VCD-трассировкой и ассемблером.
- Адаптирована и собрана ОС xv6, корректно работающая на Verilog-модели; драйвер диска заменён прямым доступом к RAM-диску.
- TinyCC портирован и интегрирован в образ xv6, обеспечивая внутрисистемную компиляцию.
- Работоспособность подтверждена тестами; продемонстрирован полный сквозной цикл.
- Обеспечена воспроизводимость: Docker, Makefile, Git-репозиторий.
- Учебно-методические материалы готовы к внедрению в курсы архитектуры ЭВМ и системного программирования.